

Mastering JavaScript Promise Methods: `all`, `allSettled`, `any`, and `race`

What Are Promises?

A **Promise** in JavaScript is an object that represents the eventual result (or failure) of an **asynchronous** operation.

We often need to deal with **multiple promises at once**, and JavaScript provides **four powerful static methods** to manage them:

```
Promise.all()
Promise.allSettled()
Promise.any()
Promise.race()
```

Let's Use the Same Real-World Example for All

Scenario:

You're fetching weather data from 3 different servers. These functions simulate the server responses:

```
const serverA = () =>
  new Promise((resolve) => setTimeout(() => resolve("A: Sunny"), 300));

const serverB = () =>
  new Promise((resolve, reject) => setTimeout(() => reject("B: Failed"), 200));

const serverC = () =>
  new Promise((resolve) => setTimeout(() => resolve("C: Cloudy"), 100));
```

Promise.all()

Use When:

All promises must succeed.

Behavior:

- Waits for **all** promises to **fulfill**
- **Rejects** immediately if **any** promise fails
- Returns an **array** of results in the input order

Example:

```
Promise.all([serverA(), serverB(), serverC()])  
  .then(results => console.log("All results:", results))  
  .catch(error => console.error("One failed:", error));
```

Output:

```
One failed: B: Failed
```

Return Format:

-  On success: ["A: Sunny", "B: ...", "C: Cloudy"]
-  On failure: First error (not an array)

Promise.allSettled()

Use When:

You want to know the result of **all** promises, regardless of their success/failure.

Behavior:

- Waits for **all** promises to **settle** (success or failure)
- **Never rejects**
- Returns an **array** of structured result objects

Example:

```
Promise.allSettled([serverA(), serverB(), serverC()])
  .then(results => {
    results.forEach((result, index) =>
      console.log(`Server ${index + 1}:`, result)
    );
  });
```

Output:

```
[
  { status: "fulfilled", value: "A: Sunny" },
  { status: "rejected", reason: "B: Failed" },
  { status: "fulfilled", value: "C: Cloudy" }
]
```

Return Format:

Array of `{ status, value }` or `{ status, reason }` objects

Promise.any()

Use When:

You only want the **first successful** result.

Behavior:

- Resolves with the **first fulfilled** promise
- Ignores rejections unless **all** reject
- Returns the fulfilled promise's **value only**

Example:

```
Promise.any([serverA(), serverB(), serverC()])  
  .then(result => console.log("First success →", result))  
  .catch(error => console.error("All failed:", error));
```

Output:

```
First success → C: Cloudy
```

If all failed:

```
AggregateError: All promises were rejected
```

Return Format:

-  First successful value (not an array)
-  AggregateError if all fail (includes `.errors[]` array)

Promise.race()

Use When:

You want the **first result that settles** (success or failure).

Behavior:

- Settles (resolves or rejects) as soon as the **first promise settles**
- Result can be success or failure
- Returns a **single value or error**

Example:

```
Promise.race([serverA(), serverB(), serverC()])  
  .then(result => console.log("First result →", result))  
  .catch(error => console.error("First rejected →", error));
```

Output:

```
First result → C: Cloudy
```

Or:

```
First rejected → B: Failed (if it finishes first)
```

Return Format:

-  First resolved value
-  First rejected reason

Comparison Table

Method	When It Resolves	Return Type	Rejects When	Good For
<code>Promise.all()</code>	All fulfilled	✅ Array of values	❌ Any fails	Gathering multiple required results
<code>Promise.allSettled()</code>	All settled (success/failure)	✅ Array of objects	❌ Never	Get status of every task
<code>Promise.any()</code>	First fulfilled	❌ Single value	❌ All fail	Use fallback options, first success
<code>Promise.race()</code>	First settled (success/failure)	❌ Single value/error	❌ First is rejection	First completed result or timeout

Sample Interview Questions

✅ Conceptual

- What's the difference between `Promise.all()` and `Promise.allSettled()`?
- When would you use `Promise.any()` instead of `Promise.race()`?
- What happens when one promise in `Promise.all()` fails?

✅ Practical / Coding

1. // Fetch from 3 sources; return the first one that works

```
function getFirstAvailableWeather() {
  return Promise.any([serverA(), serverB(), serverC()]);
}
```
2. // Log success/failure for all requests

```
function logWeatherResults() {
  return Promise.allSettled([serverA(), serverB(), serverC()])
    .then(results => results.forEach(console.log));
}
```

```

3. // Add timeout wrapper using Promise.race
function withTimeout(promise, ms) {
  const timeout = new Promise( (_, reject) =>
    setTimeout(() => reject("Timed out"), ms)
  );
  return Promise.race([promise, timeout]);
}

```

Learning Tips

- Practice combining these in real API workflows
- Always handle promise exceptions with `.catch()` or `try/catch` when using `await`
- Use DevTools (`console.time()`) to measure and compare outcomes

Summary

Method	Returns	Returns in Array?	Rejects?	Best Use
<code>all</code>	All values	✅ Yes	❌ If any fail	Required multiple completions
<code>allSettled</code>	All statuses	✅ Yes (objects)	❌ Never	Log/report all outcomes
<code>any</code>	First success	❌ No	❌ If all fail	First success/fallback logic
<code>race</code>	First settle	❌ No	❌ If first rejects	First to respond/timeout